Control Flow & Conditionals

Module 1: Python Foundations

```
temperature = 71
if temperature > 80:
    print("Too hot")
else:
    print("Just right")
```

By the End of Today, You'll Be Able To…

1

Trace a snippet and predict its output

2

Define conditional, loop, and iteration

You'll Know These by the End of Today

If / Else

decision

Loop

repeat

Iteration

one pass

Loop body

repeated code

`range()`

numbers to loop over

CHECK FOR UNDERSTANDING

Quick poll: Is `3.14` an `int`, a `float`, or a `string`?

A `int`

B `string`

C `float`

ANSWER KEY

Module 0 Data-Type Recall

Correct answer: `float`

A number with a decimal point is a `float`. `int` is whole numbers only; `string` is text in quotes.

Now Make It Do Something

Knowing what a variable is and having code actually decide something because of it are two different skills — today's the second one.

A Conditional Is a Decision

`if`

checks first

A Conditional Is a Decision

`if`

checks first

`elif`

checks next, only if needed

A Conditional Is a Decision

`if`

checks first

`elif`

checks next, only if needed

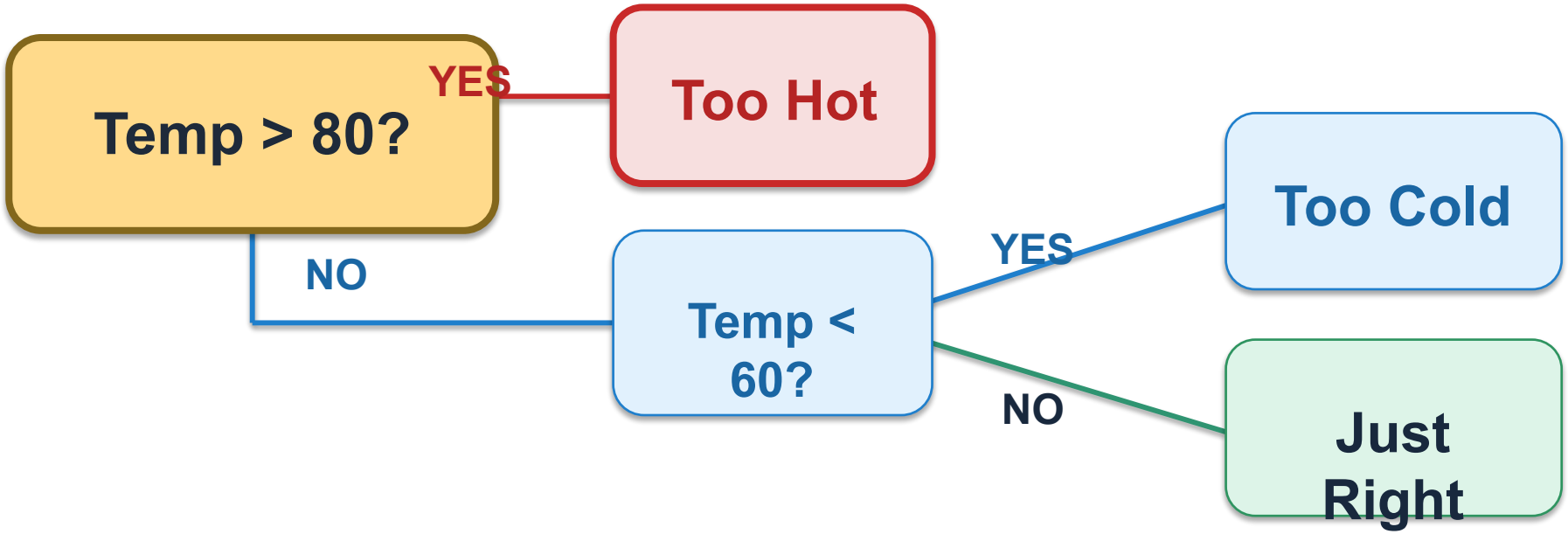
`else`

catches everything else

Traffic-light analogy

Green: go • Yellow: slow down • Red: stop
Only one path happens.

If / Elif / Else Picks Exactly One Path



Python stops after the first true branch.

Code Along: If / Elif / Else

Reading the rule is one thing. Typing it and watching Python pick exactly one branch is what makes it stick.

If/Elif/Else, Live

```python
temperature = 71

if temperature > 80:
    print("Too hot")
elif temperature < 60:
    print("Too cold")
else:
    print("Just right")
```

A Loop Is Repetition

`for`

repeats once per item

`while`

repeats until false

iteration

one pass through the loop

Dish-stack analogy

Wash one dish, then repeat for the next, then stop when the stack is empty.

Code Along: For Loop

Reading it is easy. Typing it and watching each iteration happen is what makes counting iterations click.

For Loop, Live

python

```python
scores = [88, 92, 75, 100]

for score in scores:
    print(score)
```

Code Along: The `range()` Function

Sometimes there's no list yet — you just want to repeat something a specific number of times.

`range()` Generates Numbers to Loop Over

python

```python
for i in range(5):
    print(i)
```

CHECK FOR UNDERSTANDING

What does this loop print?

```
for i in range(3):
    print(i)
```

A

1 2 3

B

0 1 2

C

1 2 3 4

Predict the output before running it.

ANSWER KEY

Loop Trace

Correct answer: 0 1 2

`range(3)` produces 0, 1, 2 — three values starting at 0 and stopping before 3. This is the classic off-by-one trap.

ADD TO THE BOARD

Put a While Loop's Steps in Order

A

Run the loop body

B

Check the condition again

C

Check the condition the first time

Two of these steps repeat until the condition is false.

Code Along: While Loop

A while loop repeats by condition instead — same idea, different trigger for stopping.

While Loop, Live

```python
count = 0

while count <= 3:
    print(count)
    count = count + 1
```

Watch It Run: The While Loop

Step 1 of 7

count starts at 0

```
count = 0
while count <= 3:
    print(count)
    count = count + 1
```

OUTPUT

ANIMATED IN CLASS: watch it run in the lecture recording

CHECK FOR UNDERSTANDING

If `count = 0` and the loop runs while `count < 3`, how many numbers print?

A

3 (0, 1, 2)

B

4 (0, 1, 2, 3)

C

2 (0, 1)

ANSWER KEY

While-Loop Off-by-One

Correct answer: 3 (0, 1, 2)

`count < 3` stops being true once `count` reaches 3, so 3 never prints. Using `<=` would print four values.

Code Along: Loop + Conditional

A loop visits every item; a conditional decides what to do with each one. This combination powers the functions ahead.

Loop and Conditional, Together

python

```python
scores = [88, 92, 75, 100]

for score in scores:
    if score >= 90:
        print(score, "- great job")
    else:
        print(score, "- keep practicing")
```

Watch It Run: Loop + If/Else

Step 1 of 6

scores holds four numbers

```
scores = [88, 92, 75, 100]
for score in scores:
    if score >= 90:
        print(score, "- great job")
    else:
        print(score, "- keep practicing")
```

OUTPUT

ANIMATED IN CLASS: watch it run in the lecture recording

QUICK CHALLENGE

Your trace is wrong for one item. What's the most likely cause?

A The list had a syntax error

B Python skipped an iteration randomly

C Miscounted when a condition first became true

ANSWER KEY

Off-by-One Trace

Correct answer: Miscounted the boundary

Python doesn’t skip iterations randomly. Re-count from the first iteration, one at a time, and check the condition at each boundary.

TIP

“Walk Me Through What Prints for [95, 60, 88]”

A

95 — great job
60 — keep practicing
88 — keep practicing

B

95 — great job
60 — keep practicing
88 — great job

C

All three print `"great job"`

Use the “Loop and Conditional, Together” code. Answer like an interview, with the Code Walkthrough: inputs, control flow, state, output, edge case, improvement.

ANSWER KEY

Interview-Style Trace

Correct: 95 great job; 60 keep practicing; 88 keep practicing

Only 95 clears the `>= 90` check. 88 is close, but still below the threshold.

MODEL ANSWER

Model Answer: Code Walkthrough

INPUTS & CONTROL FLOW

Three scores; the loop checks `score >= 90` on each pass.

STATE & OUTPUT

95 prints great job; 60 and 88 print keep practicing.

EDGE CASE

88 is close, but still below the 90 cutoff.

IMPROVEMENT

Name the 90 as a `threshold` variable so it's easy to change.

Key Terms — Recap

If / Else

Choose one path

Loop

Repeat work

Iteration

One pass

Loop body

Code that repeats

`range()`

Generate loop values

So Far, We've Covered…

Already

Variables and data types
from Module 0

Today

Conditionals and loops

Next

Nested loops

KEEP LEARNING

Where to Go From Here

Python Tutor

Visualize Code Execution

Practice • 5–10 min
Step through any snippet, line by line.

Python Documentation

Conditionals

Docs • 10 min to skim
The official reference, straight from Python.